Design Justification Report

Fused Softmax + Layer Normalization Accelerator

ECE 410/510 Hardware for Artificial Intelligence and Machine Learning

Spring 2026 | Portland State University

Mahesh Chowdary Naidu

Target Platform	SKY130 HD open-source PDK
Clock Target	100 MHz (10 ns period)
Interface	AXI4-Lite control + AXI4-Stream 64-bit data
Precision	INT8 (hardware) / FP64 (software baseline)
Synthesis Cells	132 cells (Yosys 0.9, hash 9c2bb123ba)
Co-simulation	PASS 8/8 beats (iverilog 12.0)
Projected Speedup	1,466x over 60.05 ms CPU baseline (PROJECTED)

1. Problem and Motivation

Modern transformer language models spend a disproportionate fraction of their compute time in element-wise normalization operations. Profiling the professor's reference implementation — a pure NumPy transformer with configuration d=64, n_heads=4, d_ff=256, n_layers=2, T=64, B=8 running on an Intel i5-5300U at 2.30 GHz — revealed that **softmax consumed 21% of total runtime** and **layer_norm_forward consumed 7%**, together accounting for **28% of the full forward pass**. The baseline forward pass averaged **60.05 ms** at **133.2 samples/sec** with a peak memory footprint of 13.8 MB.

These two operations share a critical structural property: both require a streaming reduction over a row of d=64 INT8 values followed by per-element normalization. Executing them as separate kernels requires six memory passes over the data (read input, write exp, read exp, write softmax, read softmax, write layernorm). Fusing them into a single hardware pipeline reduces this to one read pass and one write pass, changing the effective arithmetic intensity from 0.271 FLOP/byte (unfused, software) to 5.56-11.13 FLOP/byte (fused, hardware) — a 20-40x improvement in memory efficiency.

Custom hardware is justified because: (1) the CPU cannot exploit streaming INT8 arithmetic on this workload efficiently, operating at only 6.38 GFLOP/s against a theoretical peak of 37 GFLOP/s; (2) the fused pipeline fits naturally into a fixed-latency 8-stage register-based datapath with no on-chip SRAM required; and (3) the SKY130 target platform can execute the full fused kernel in 8 clock cycles per row, projected at 195,312 samples/sec at 100 MHz.

Metric	Value	Source
Baseline execution time	60.05 ms avg, ±4.22 ms	profile_m1.py, 10 runs
Baseline throughput	133.2 samples/sec	Measured, CF09 re-run
Baseline memory	13.8 MB peak RSS	tracemalloc
Softmax share	21% of runtime	M1 profiling
LayerNorm share	7% of runtime	M1 profiling
Combined target share	28% of runtime	M1 profiling

Table 1. Software baseline measurements on DESKTOP-KDU44VU (i5-5300U, 4 GB RAM, Windows 64-bit).

2. Roofline Analysis

The roofline model was applied to the fused Softmax+LayerNorm kernel on the SKY130 HD target platform at 100 MHz. Platform parameters: peak compute approximately 1 GOPS (one 8-bit operation per cycle at 100 MHz), peak memory bandwidth approximately 0.8 GB/s (AXI4-Stream 64-bit interface at 100 MHz). The ridge point is 1.0 GOPS / 0.8 GB/s = **1.25 FLOP/byte**.

Arithmetic intensity was computed analytically from first principles (CF09 CMAN analysis). For one invocation over a row of d=64 INT8 elements the total FLOPs are: S2 (64 comparisons) + S4 (72 additions) + S5 (128 multiply+divide) + S6 (192 Welford mean ops) + S7 (256 Welford M2 ops) = **712 FLOPs per row**. S3 exp LUT lookups are counted as 0 FLOPs (ROM reads, not arithmetic).

Bound	Formula	Bytes	AI (FLOP/byte)
No-reuse (lower)	64 read + 64 write	128 bytes	712/128 = 5.56
Full-reuse (upper)	64 write only	64 bytes	712/64 = 11.13

Table 2. Arithmetic intensity bounds for fused Softmax+LayerNorm, d=64, INT8.

Both AI bounds (5.56 and 11.13 FLOP/byte) sit to the **right** of the ridge point (1.25 FLOP/byte), placing the kernel firmly in the **compute-bound region** of the SKY130 roofline. This means arithmetic units, not memory bandwidth, are the binding constraint. The roofline analysis directly shaped the architecture: since bandwidth is not the bottleneck, the design uses a simple streaming pipeline with no data reuse buffers rather than a banked SRAM with complex scheduling. See Figure 1 (bench/roofline_final.png) for the full plot.

The primary bottleneck identified through synthesis is the 8x integer division cells in Stage 5 (softmax normalize), which create a critical path of approximately 9-13 ns — marginal at 100 MHz and failing at 200 MHz. Replacing division with reciprocal multiply is the highest-leverage optimization, expected to reduce the critical path to approximately 2.5 ns and enable 200 MHz closure.

3. Precision and Data Format

The hardware accelerator operates in **INT8 precision** throughout the pipeline. Each AXI4-Stream beat carries 8 elements of 8 bits each (64-bit total bus width), matching the d=64 row dimension with 8 beats per row. The software baseline operates in FP64 (NumPy default float64), giving a 64-bit to 8-bit quantization ratio of 8:1.

INT8 was selected for three reasons: (1) it matches the target AXI4-Stream bus width exactly — 8 bytes per beat with no packing overhead; (2) INT8 arithmetic requires significantly less silicon area than FP64 multipliers and adders on SKY130; and (3) the quantization error for softmax+layernorm at INT8 precision is acceptable for inference workloads — the mean absolute error verified in M2 was MAE = 0.013 (5.1% of the INT8 range), confirmed by direct comparison against the FP64 reference.

The exp LUT approximation (8-entry lookup table, exp(-k/8)*255 for k=0..7) introduces additional approximation error in Stage 3. This was accepted because: the LUT covers the stable softmax region after max subtraction; the 8-bit output range (105-255) captures the relative magnitude differences needed for softmax normalization; and the downstream layernorm in Stages 6-8 re-normalizes the output, reducing sensitivity to exp approximation error.

Parameter	Hardware	Software Baseline
Precision	INT8 (8-bit integer)	FP64 (64-bit float)
Bus width	64-bit (8 x INT8)	64-bit per element
Exp method	8-entry LUT	numpy.exp()
Quantization MAE	0.013 (verified M2)	0 (reference)
Bits per element	8	64
Memory per row	64 bytes	512 bytes

Table 3. Precision and data format comparison.

4. Dataflow and Architecture

The accelerator implements a **no-local-reuse streaming dataflow**: each input element is read exactly once from the AXI4-Stream interface, processed through all 8 pipeline stages, and written once to the output stream. There is no weight matrix, no input buffer, and no on-chip SRAM. This pattern is appropriate because the fused softmax+layernorm kernel has no data reuse opportunity — every element is processed independently once the running statistics (max, sum, mean, variance) have been accumulated.

The 8-stage pipeline processes one 64-bit beat (8 INT8 elements) per clock cycle after the 8-cycle fill latency. All pipeline stages operate in parallel on different beats of the same or different rows:

Stage	RTL Signal	Operation	Latency
S1	pipe_data[0]	Input latch + AXI4-Stream back-pressure	1 cycle
S2	running_max	Online max tracking (numerically stable softmax)	1 cycle
S3	exp_lut	Exp approximation via 8-entry ROM lookup	1 cycle
S4	running_sum, final_sum	Softmax denominator accumulation (24-bit)	1 cycle
S5	norm_beat	Softmax normalize: (exp * 255) / final_sum	1 cycle
S6	welford_mean	Welford online mean for LayerNorm	1 cycle
S7	welford_m2	Welford online variance (M2 accumulator)	1 cycle
S8	pipe_data[7]	LayerNorm output (g=1, b=0 pass-through)	1 cycle

Table 4. Pipeline stage breakdown. Total latency = 8 cycles. Throughput = 1 row per 8 cycles after fill.

The pipeline registers (pipe_data[0..7], pipe_valid[0..7], pipe_last[0..7]) are all synchronous with asynchronous active-low reset (rst_n). The final_sum register added in M4 fixes a Verilog non-blocking assignment race where running_sum was reset to zero on the same clock edge that Stage 5 needed it for normalization of the last beat. The fix captures the complete row sum into final_sum when pipe_last[3] fires, one stage before the reset clears running_sum. See Figure 2 for the block diagram.

5. Hardware Interface

Two industry-standard interfaces were implemented, both selected in M1 and unchanged through M4:

AXI4-Lite Control Interface

A 6-bit address, 32-bit data AXI4-Lite slave register bank provides host CPU control. Five registers are implemented: CTRL (0x00, start and soft-reset), STATUS (0x04, read-only done flag), CFG_D (0x08, row width), CFG_T (0x0C, number of rows), and PRECISION (0x10, INT8/FP64 select). All five AXI4-Lite channels (AW/W/B/AR/R) are implemented with full handshake compliance. The write channel accepts AWVALID+WVALID simultaneously for single-beat writes; BVALID is asserted one cycle later.

AXI4-Stream Data Interface

A 64-bit AXI4-Stream slave (input) and master (output) carry the activation data. TDATA is 8 bytes wide (8 x INT8 elements per beat), TVALID/TREADY back-pressure is honored, and TLAST marks the last beat of each row. The effective bandwidth at 100 MHz is **64 bits x 100 MHz = 800 MB/s**. At the projected throughput of 195,312 samples/sec with 64 bytes per row and 64 rows per sample, the required bandwidth is 195,312 x 64 x 64 = 800 MB/s — exactly matching the interface capacity. The design is interface-bound at the projected throughput.

Interface	Width	Bandwidth at 100 MHz	Purpose
AXI4-Lite	32-bit data, 6-bit addr	~100 MB/s (control only)	Config + status
AXI4-Stream in	64-bit TDATA	800 MB/s	Input activations
AXI4-Stream out	64-bit TDATA	800 MB/s	Output activations

Table 5. Interface summary.

6. Verification

Correctness was verified in three stages across M2 and M3, with the final end-to-end simulation repeated for M4.

M2 Unit Verification

compute_core.sv was verified with 5 directed test cases covering: (1) single-beat softmax with known reference, (2) multi-beat row with monotonically increasing values, (3) uniform input (all elements equal — tests denominator accumulation edge case), (4) back-pressure handling (m_axis_tready deasserted mid-row), and (5) consecutive rows without reset (tests running state clearing). All 5 cases produced PASS. interface.sv was separately verified with 8 test cases covering all AXI4-Lite register addresses and AXI4-Stream handshake sequences, producing 8/8 PASS.

M3 End-to-End Co-Simulation

The integrated top.sv (interface_mod_m3 + compute_core_m3) was verified with an end-to-end testbench that drives the design exclusively through AXI4-Lite configuration writes and AXI4-Stream data beats — no direct access to compute_core ports. The testbench sends 8 beats of d=64 INT8 data (one complete row), collects 8 output beats, and verifies all outputs are non-zero. Result: **PASS 8/8 beats**. The M3 simulation log is committed at project/m3/sim/cosim_run.log.

M4 Final Simulation

The M4 RTL (renamed compute_core.sv, interface.sv, top.sv with corrected module names) was re-simulated with the same testbench. A bug discovered during M3 integration — a Verilog non-blocking assignment race in the running_sum/final_sum logic causing beat 7 to output all zeros — was fixed by adding the final_sum register. The M4 simulation produces **PASS 8/8 beats**. The final simulation log is at project/m4/sim/final_run.log.

Milestone	Test	Result	Log
M2	compute_core unit (5 cases)	PASS 5/5	project/m2/sim/
M2	interface_mod unit (8 cases)	PASS 8/8	project/m2/sim/
M3	End-to-end co-sim (8 beats)	PASS 8/8	project/m3/sim/cosim_run.log
M4	End-to-end final sim (8 beats)	PASS 8/8	project/m4/sim/final_run.log

Table 6. Verification summary across all milestones.

7. Synthesis Results

Synthesis was performed with Yosys 0.9 (git sha1 1979e0b) on Google Colab targeting SKY130 HD at a 10 ns (100 MHz) clock period. The full Yosys log is committed at project/m4/synth/openlane_run.log (hash 9c2bb123ba). Full OpenLane 2 place-and-route was not completed due to RAM constraints (host machine: 4 GB; OpenLane 2 requires >8 GB). Chip area is reported as 0 because SKY130 liberty cells were not mapped in Yosys 0.9 generic mode; area is estimated manually.

Cell Count

Cell Type	Count	Function	Est. Area
$adff	39	Pipeline registers + AXI regs (async active-low reset)	312 um^2
$mux	43	Pipeline selects + AXI back-pressure muxes	172 um^2
$div	8	S5 softmax normalize — CRITICAL PATH bottleneck	2400 um^2
$mul	8	S5 multiply (exp * 255)	400 um^2
$add	8	S4 beat_sum accumulator tree	160 um^2
$gt	9	S2 running_max comparators	135 um^2
$eq	7	AXI4-Lite address decode	70 um^2
Other	10	Logic, ROM, reduce	40 um^2
Total	132	—	~3,689 um^2

Table 7. Cell breakdown from Yosys stat pass. Area estimated from SKY130 HD typical cell sizes.

Timing

The Yosys LTP pass reported a longest topological path of **275 nodes**, from rst_n (source) to pipe_data[7][0] (= m_axis_tdata[0], sink). The critical combinational path per clock cycle runs from the final_sum DFF output through the $ne zero-check, 8 parallel $div cells, and a $ternary mux to the pipe_data[4] DFF input. Estimated delay: **9-13 ns**. The design is marginal at 100 MHz and fails timing at 200 MHz. Formal setup/hold slack requires OpenSTA with SKY130 liberty (planned for future work).

Power

Full OpenLane 2 power estimation was not available. Manual estimates from SKY130 HD datasheet values give: static leakage ~963 nW (dominated by 8x $div cells at 100 nW each); dynamic power ~64.2 uW at 100 MHz with activity factor 0.3; total **~65.1 uW**. Energy per sample: 65.1 uW x 5.12 us = **0.333 nJ** (PROJECTED). See project/m4/synth/power_report.txt.

8. Benchmark Results

Benchmark numbers are **PROJECTED** from synthesis cycle counts and target clock frequency per the M4 spec fallback path. No FPGA or cocotb end-to-end measurement was available. All projected numbers are labeled as such throughout this report and in project/m4/bench/benchmark.md and benchmark_data.csv.

Metric	SW Baseline (measured)	HW Accelerator (PROJECTED)
Platform	i5-5300U @ 2.30 GHz	SKY130 HD @ 100 MHz
Precision	FP64	INT8
Execution time	60.05 ms	5.12 us (PROJECTED)
Throughput	133.2 samples/sec	195,312 samples/sec (PROJECTED)
Speedup	1x (baseline)	1,466x (PROJECTED)
Power	~15 W (TDP)	~65.1 uW (PROJECTED)
Energy/sample	900.75 mJ	0.333 nJ (PROJECTED)
Energy improvement	1x	~2.7 x 10^9 x (PROJECTED)

Table 8. Benchmark comparison. SW baseline re-measured on DESKTOP-KDU44VU for CF09. Raw data in bench/benchmark_data.csv.

The projected speedup of 1,466x vs the CF09 re-run baseline (133.2 samples/sec) reflects the architectural advantage of dedicated streaming hardware over a general-purpose CPU executing interpreted NumPy operations. The dominant uncertainty is timing closure: if the $div critical path forces the clock down to 50 MHz rather than 100 MHz, the speedup falls to 733x, still a substantial improvement. The second uncertainty is AXI4-Stream back-pressure: the projection assumes one beat accepted per cycle with no stalls.

The energy improvement of approximately 2.7 billion times reflects the difference between a 15W CPU running for 60 ms and a 65 uW accelerator running for 5.12 us. Even accounting for uncertainty in the power estimate (factor of 2-3x), the energy efficiency advantage of dedicated INT8 hardware over FP64 CPU remains at least 10^8 times. This is consistent with published results for ASIC accelerators vs CPU baselines on similar workloads.

For reference against the original M1 baseline (17.25 ms, 463.9 samples/sec), the projected speedup is 421x (195,312 / 463.9). The difference between 421x and 1,466x reflects the variation in CPU measurement between M1 (warmed-up steady-state) and CF09 (immediately after a 500-step training run). Both numbers are honest measurements on the same hardware. See Figure 3 for the roofline plot.

9. What Did Not Work

This section documents the three most significant failures and the lessons learned from each.

9.1 Integer Division Timing Closure Failure

The original M2 design used the Verilog division operator ({16'd0,eb} * 24'd255) / running_sum in Stage 5 without considering that integer division synthesizes to an iterative multi-cycle circuit. Yosys 0.9 maps each / operator to a $div cell requiring approximately 24 gate levels for a 24-bit divisor. On SKY130 HD at TT/25C/1.8V, each $div cell is estimated at 8-12 ns — exceeding both the 5 ns (200 MHz) and 10 ns (100 MHz) clock periods. The design does not close timing at either target frequency. This was discovered at first synthesis in CF07/M3 and not fixed by M4 submission time. The fix (reciprocal multiply: recip = 65536 / final_sum, norm = (eb * recip) >> 8) was identified and documented in remaining_tasks.md but not implemented due to time constraints. Lesson: synthesize early and never use / or % in RTL on a critical path.

9.2 Verilog Non-Blocking Assignment Race in running_sum

The M2 and initial M3 compute_core had a subtle RTL bug: in the S4 always block, two non-blocking assignments to running_sum fired on the same clock edge when pipe_last[3] was high — running_sum <= running_sum + beat_sum and running_sum <= 24'd0 (reset). In Verilog, the last non-blocking assignment in program order wins, so the reset always overwrote the accumulation result. This caused beat 7 (the final beat of every row, which carries tlast=1) to see running_sum=0 at Stage 5 and produce all-zero normalized output. The bug was not caught in M2 unit tests because those tests used shorter rows or did not check the last beat specifically. It was caught in M3 co-simulation when beat 7 output was observed as 0x0000000000000000. The fix added a final_sum register that captures running_sum + beat_sum at the pipe_last[3] boundary before the reset fires, and Stage 5 normalizes against final_sum rather than running_sum.

9.3 OpenLane 2 Full Place-and-Route Not Completed

Full OpenLane 2 place-and-route was planned for M3 and M4 to obtain mapped cell area in um^2, formal setup/hold slack from OpenSTA with SKY130 liberty, and dynamic power from OpenROAD with VCD activity annotation. All three of these would convert estimated numbers to measured numbers. The flow was not completed because the host machine (Intel i5-5300U, 4 GB RAM) cannot run the OpenLane 2 Docker image, which requires a minimum of 8 GB RAM. Colab free tier provides Yosys but not the full OpenLane 2 environment. As a result, the chip area reports 0.000000 um^2, timing slack is unavailable, and the power estimate is manual. The PSU research computing cluster has sufficient RAM and was identified as the path forward, but cluster access was not obtained within the project timeline. If given additional time, this would be the first action taken.

Failure	Impact	Fix / Lesson
$div timing closure	Design fails 100/200 MHz	Replace with reciprocal multiply

running_sum NB race	Beat 7 output all zeros	Added final_sum register
OpenLane 2 RAM limit	Area=0, no formal STA	Use PSU cluster (>8 GB)

Table 9. Summary of failures and lessons learned.

Figure References

Figure 1 — Final Roofline Plot. File: project/m4/bench/roofline_final.png. Shows SKY130 roofline (100 MHz), i5-5300U roofline, SW baseline measured point, HW accelerator projected point, and kernel AI range [5.56, 11.13] FLOP/byte. Referenced in Sections 2 and 8.

Figure 2 — Pipeline Block Diagram. File: project/m4/report/figures/block_diagram.png. Shows 8-stage pipeline (S1-S8), AXI4-Lite register bank, AXI4-Stream interfaces, and final_sum register. Referenced in Section 4.

Figure 3 — Final Waveform. File: project/m4/sim/final_waveform.png. GTKWave screenshot showing Region 1 (AXI4-Lite config writes), Region 2 (AXI4-Stream input, 8 beats), Region 3 (AXI4-Stream output, 8 beats, all non-zero). Referenced in Section 6.

Figure 4 — Synthesis Critical Path. File: project/m4/report/figures/critical_path.png. Dataflow diagram showing final_sum DFF to pipe_data[4] DFF through 8x $div cells (LTP nodes 268-272, 275-node total path). Referenced in Section 7.